

Guía rápida de Python

Lo esencial para Estructura de Datos

INF 222 - Estructura de Datos - Semestre 2026-2

Material de refuerzo -- no reemplaza el contenido oficial del curso

Índice: 1) Variables y tipos 2) Operadores 3) Control de flujo 4) Funciones 5) Colecciones 6) Manejo de errores 7) Programación orientada a objetos 8) Recursión 9) Buenas prácticas 10) Sitios para seguir aprendiendo

1. Variables y tipos de datos

Python no declara tipos: el tipo se infiere del valor asignado. Una misma variable puede cambiar de tipo (evita hacerlo a propósito, pero es válido).

```
edad = 20          # int
altura = 1.75      # float
nombre = "Ana"     # str
activo = True      # bool
nada = None        # ausencia de valor
type(edad)         # <class 'int'>
type(edad).__name__ # "int"
```

Conversión entre tipos

```
int("5")          # 5
float("3.2")       # 3.2
str(42)            # "42"
int("abc")         # ValueError: invalid literal for int()
```

2. Operadores

Tipo	Operadores	Ejemplo
Aritméticos	+ - * / // % **	7 // 2 -> 3 7 % 2 -> 1 2**10 -> 1024
Comparación	== != < > <= >=	5 == 5 -> True
Lógicos	and or not	edad >= 18 and activo
Pertenencia	in / not in	3 in [1, 2, 3] -> True
Asignación	= += -= *= /=	contador += 1

f-strings (formatear texto)

```
nombre = "Ana"
edad = 20
print(f"{nombre} tiene {edad} años")
print(f"El doble es {edad * 2}")
```

3. Control de flujo

if / elif / else

```
if edad < 13:
    categoria = "niño"
elif edad < 18:
    categoria = "adolescente"
else:
    categoria = "adulto"
```

Python usa indentación (4 espacios) en vez de llaves {} para marcar bloques. Un error de indentación es un error de sintaxis, no de estilo.

while (repite mientras la condición sea verdadera)

```
i = 1
while i <= 5:
    print(i)
    i += 1    # sin esto: bucle infinito
```

for (recorre una secuencia)

```
for i in range(5):           # 0,1,2,3,4
    print(i)

for elemento in [10, 20, 30]:
    print(elemento)

for i, valor in enumerate(["a", "b"]):
    print(i, valor)          # 0 a, 1 b
```

break / continue

```
for x in lista:
    if x == buscado:
        break          # sale del bucle ya

for x in lista:
    if x < 0:
        continue      # salta al siguiente x
    print(x)
```

4. Funciones

```
def es_par(numero):
    return numero % 2 == 0

def saludar(nombre, saludo="Hola"): # valor por defecto
    return f"{saludo}, {nombre}!"

def sumar(*numeros):                # args variables
    return sum(numeros)

sumar(1, 2, 3)      # 6
saludar("Ana")      # "Hola, Ana!"
```

Una función puede recibir otra función

```
def aplicar(lista, operacion):
    return [operacion(x) for x in lista]

aplicar([1, 2, 3], lambda x: x * 2) # [2, 4, 6]
```

5. Colecciones nativas

Tipo	Sintaxis	Modificable	Uso típico
list	[1, 2, 3]	Sí	Secuencia que cambia (Pila, Cola)
tuple	(x, y)	No	Datos fijos, ej. coordenadas
dict	{"k": v}	Sí	Buscar por clave
set	{1, 2, 3}	Sí	Pertenencia rápida, sin duplicados

Métodos de listas más usados

```
p = []
p.append(10)    # agrega al final -- O(1)
p.pop()         # quita y retorna el último -- O(1)
p.insert(0, 99) # inserta en una posición -- O(n)
p[-1]          # último elemento
p[1:3]         # slicing: sub-lista
len(p)         # tamaño
```

Diccionarios

```
frecuencias = {}  
frecuencias["a"] = frecuencias.get("a", 0) + 1  
for clave, valor in frecuencias.items():  
    print(clave, valor)
```

6. Manejo de errores

```
def pop(self):
    if self.is_empty():
        raise IndexError("pop en pila vacía")
    return self._datos.pop()

try:
    pila.pop()
except IndexError:
    print("La pila estaba vacía")
```

Excepciones comunes

Excepción	Cuándo ocurre
IndexError	Índice fuera de rango en una lista
KeyError	Clave inexistente en un diccionario
ValueError	Valor de tipo correcto pero inválido, ej. <code>int("abc")</code>
TypeError	Operación entre tipos incompatibles
ZeroDivisionError	División entre cero

7. Programación orientada a objetos

`self` es el objeto sobre el que se llamó el método; Python lo pasa automáticamente. `__init__` es el constructor: corre una vez, al crear el objeto.

```
class Pila:
    def __init__(self):
        self._datos = []          # atributo

    def push(self, dato):          # método
        self._datos.append(dato)

    def pop(self):
        if not self._datos:
            raise IndexError("pila vacía")
        return self._datos.pop()

    def __str__(self):             # usado por print()
        return f"Pila: {self._datos}"

p = Pila()
p.push(10)
print(p)                          # Pila: [10]
```

8. Recursión

```
def funcion(problema):
    if es_caso_base(problema):    # 1. CASO BASE
        return valor_directo
    return combinar(              # 2. CASO RECURSIVO
        parte, funcion(resto_del_problema)
    )

def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

Sin caso base: recursión infinita -> `RecursionError`. Usa pythontutor.com para ver, paso a paso, cómo se apilan las llamadas.

9. Buenas prácticas rápidas (PEP 8)

- Nombres de variables y funciones: snake_case (minúsculas_con_guion_bajo).
- Nombres de clases: PascalCase (PrimeraLetraMayúscula), ej. ListaEnlazada.
- Indentación: 4 espacios, nunca mezclar con tabs.
- Un solo import por línea; imports arriba del archivo.
- Nombres descriptivos: 'contador' en vez de 'c', 'lista_numeros' en vez de 'l'.
- Comentarios: explican el POR QUÉ, no el QUÉ (el código ya dice qué hace).

10. Sitios reales para seguir aprendiendo

Sitio	Para qué sirve
docs.python.org/es/3/tutorial	Tutorial oficial de Python, en español
pythontutor.com	Visualiza paso a paso la ejecución y la memoria
w3schools.com/python	Referencia corta con editor interactivo
realpython.com	Artículos y tutoriales más profundos
programiz.com/python-programming	Lecciones cortas, buen repaso pre-parcial
py4e.com	Curso completo Python for Everybody, gratuito
cs50.harvard.edu/python	Curso universitario completo (Harvard/edX)
exercism.org/tracks/python	Ejercicios cortos con retroalimentación

Siguiente paso: resuelve los laboratorios lab00 a lab05 en [refuerzo-python/laboratorios/](#), en orden, corriendo pytest en cada uno.